transactions

Remy Wang

April 2026

So far we've been treating DBs as inanimate objects: they hold data, we can look at the data, and that's it. But SQL databases are so popular because they deal with *changes* very well. Let's see some examples.

I'm in a good mood today and decided to give everyone free points! I vibed a quick python script to do this:

UID	score
1	39
2	24
3	40
4	35
...	...

I'm in a good mood today and decided to give everyone free points! I vibed a quick python script to do this:

UID	score
1	39
2	24
3	40
4	35
...	...

Uh oh! My computer crashed halfway through, and I don't know who still haven't got the free points!

You don't like that, because some of you got the points and some didn't. You would be fine if either none of you get the points or all of you did.

Let's try another experiment. This time, your fate is in your own hands: steal points from your enemies! Subtract some points from someone and add it to your own. Here's the Google sheet, go!

Well, that was a mess. The total score no longer add up, meaning our DB is not *consistent*!

The problem is that you were *interfering* with each other!

Well, that was a mess. The total score no longer add up, meaning our DB is not *consistent*!

The problem is that you were *interfering* with each other!

One solution is to go *one at a time*, so that every change is made in *isolation*. In other words, we *serialize* the changes.

Another solution is we first claim a *lock* on a piece of data before changing it. This time, wait a few seconds before editing a cell, and only edit it if no one else is trying to edit.